

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: ALB Returns 503 Service Unavailable

Interview Question

Users receive a "503 Service Unavailable" error when accessing the application through an Application Load Balancer (ALB). How would you troubleshoot it?

Answer

A **503** usually means the ALB cannot forward requests to healthy backend targets.

Step 1: Check Target Group Health

Verify whether the EC2 instances are **Healthy**.

If all targets are unhealthy, ALB returns **503**.

Step 2: Verify Health Check

Check:

- Health Check Path
- Port
- Protocol
- Success Code

Example:

```
Health Check:  
/health
```

If the application doesn't return **HTTP 200**, the instance becomes unhealthy.

Step 3: Verify the Application

SSH into the EC2 instance.

Check the application:

```
systemctl status myapp
```

Test locally:

```
curl localhost:8080/health
```

Step 4: Check Security Groups

Ensure:

- ALB Security Group allows HTTP/HTTPS.
 - EC2 Security Group allows traffic **from the ALB Security Group**.
-

Step 5: Review Logs

Check:

- Application logs
- ALB Access Logs
- CloudWatch Metrics

Look for:

- 5XX errors
 - Timeout errors
 - Application crashes
-

Real-Time Scenario

After a deployment, users received **503 Service Unavailable**. Investigation showed the application had failed to start, causing all targets to become unhealthy. Restarting the application restored the health checks, and the ALB resumed routing traffic.

Best Practices

- Use a dedicated `/health` endpoint.
 - Monitor Target Group health.
 - Enable ALB Access Logs.
 - Configure Auto Scaling to replace unhealthy instances.
-

Interview Summary

"When an ALB returns a 503 error, I first check the Target Group health, verify the health check configuration, confirm the application is running, review Security Groups, and analyze logs. In most cases, unhealthy targets or a failed application are the root cause."

Scenario 2: ALB Target Health Check Failing

Interview Question

Your EC2 instances are running, but the ALB marks them as unhealthy. How would you troubleshoot it?

Answer

If ALB health checks are failing, I verify the application, health check configuration, and network settings.

Step 1: Check Health Check Configuration

Verify:

- Health Check Path
- Port
- Protocol
- Success Code (200)

A wrong path or port is the most common cause.

Step 2: Verify the Application

SSH into the EC2 instance.

Check if the application is running:

```
systemctl status myapp
```

Test the health endpoint:

```
curl localhost:8080/health
```

The endpoint should return **HTTP 200**.

Step 3: Check Target Port

Ensure the **Target Group port** matches the application's listening port.

Example:

- Application → **8080**
 - Target Group → **8080**
-

Step 4: Check Security Groups

Verify:

- ALB Security Group allows HTTP/HTTPS.
 - EC2 Security Group allows traffic **from the ALB Security Group** on the application port.
-

Step 5: Review Logs

Check:

- Application logs
- ALB Target Health reason
- CloudWatch metrics

Look for timeout or connection errors.

Real-Time Scenario

After deploying a new application version, all EC2 instances became **Unhealthy**. The application had been moved from **port 8080 to 9090**, but the Target Group was still checking **8080**. After updating the Target Group port, the instances became healthy again.

Best Practices

- Use a lightweight `/health` endpoint.

- Keep the Target Group port aligned with the application port.
 - Monitor **HealthyHostCount** in CloudWatch.
 - Validate health checks before deployment.
-

Interview Summary

"If ALB health checks fail, I verify the health check configuration, ensure the application is running, confirm the Target Group port matches the application port, check Security Groups, and review logs. Most issues are caused by incorrect health check settings or an application that isn't responding."

Scenario 3: Network Load Balancer (NLB) Is Not Forwarding Traffic

Interview Question

Your application is behind a Network Load Balancer (NLB), but clients cannot connect. How would you troubleshoot it?

Answer

If an NLB is not forwarding traffic, I verify the target health, listener, networking, and backend application.

Step 1: Check Target Health

Verify that the registered targets are **Healthy**.

If all targets are unhealthy, the NLB cannot forward traffic.

Step 2: Verify Listener & Target Group

Ensure:

- Listener port is correct (e.g., 80, 443, 3306).
 - Target Group is forwarding to the correct backend port.
-

Step 3: Check Backend Application

Verify the application is running and listening on the expected port.

```
sudo ss -tulnp
```

or

```
netstat -tulnp
```

Step 4: Check Security Groups & NACLs

Verify:

- EC2 Security Group allows traffic from the required source.
- Network ACLs allow inbound and outbound traffic.

Note: NLB operates at **Layer 4 (TCP/UDP)** and doesn't modify the client source IP by default.

Step 5: Review Logs & Metrics

Check:

- Application logs
- CloudWatch metrics
- Target Health status

Look for connection failures or timeout errors.

Real-Time Scenario

A production database was behind an NLB on **port 3306**. Clients couldn't connect because the application had started on **3307** after a configuration change, while the Target Group still forwarded traffic to **3306**. Updating the Target Group port resolved the issue.

Best Practices

- Ensure the application listens on the correct port.
- Configure proper TCP health checks.
- Monitor HealthyHostCount and UnHealthyHostCount.
- Deploy targets across multiple Availability Zones.

Interview Summary

"If an NLB is not forwarding traffic, I first check the Target Group health, verify the listener and target ports, ensure the backend application is running, validate Security Groups and Network ACLs, and review CloudWatch metrics and application logs to identify the root cause."

💡 Interview Tip

A common follow-up question is:

What is the difference between ALB and NLB?

ALB	NLB
Layer 7 (HTTP/HTTPS)	Layer 4 (TCP/UDP/TLS)
Path & Host-based routing	No path-based routing
Supports WebSockets	Best for low latency and high throughput
SSL termination supported	TLS pass-through or termination
Best for web applications	Best for databases, gaming, IoT, MQTT

Mentioning this comparison after answering often leaves a strong impression in AWS interviews.

Scenario 4: SSL/TLS Certificate Issue on ALB

Interview Question

Users cannot access your application over HTTPS, or they receive an SSL certificate error. How would you troubleshoot it?

Answer

If HTTPS is not working on an ALB, I verify the SSL certificate, listener, DNS, and security configuration.

Step 1: Check the SSL Certificate

Verify that:

- The ACM certificate is **Issued** (not Pending or Expired).
 - The certificate matches the domain name.
-

Step 2: Check the HTTPS Listener

Ensure the ALB has:

- HTTPS Listener (Port **443**)
 - Correct SSL certificate attached
 - Correct Target Group configured
-

Step 3: Verify DNS Configuration

Check that the domain name points to the ALB using:

- Route 53 Alias Record
- CNAME (if applicable)

A wrong DNS record may direct traffic to another server.

Step 4: Check Security Groups

Verify:

- ALB Security Group allows **HTTPS (443)**.
 - EC2 Security Group allows traffic from the ALB Security Group.
-

Step 5: Test HTTPS

Run:

```
curl -Iv https://your-domain.com
```

Check:

- SSL handshake
- Certificate details
- HTTP response

Real-Time Scenario

Users reported an **SSL certificate warning** after a new domain was added. Investigation showed the ALB was using an ACM certificate that didn't include the new domain. After attaching the correct certificate, HTTPS worked successfully.

Best Practices

- Use **AWS Certificate Manager (ACM)** for SSL certificates.
 - Enable automatic certificate renewal.
 - Redirect **HTTP (80)** to **HTTPS (443)**.
 - Monitor certificate expiry.
-

Interview Summary

"If HTTPS is not working on an ALB, I verify the ACM certificate status, HTTPS listener configuration, DNS records, Security Groups, and test the SSL handshake using curl or a browser. Most issues are caused by an incorrect certificate, expired certificate, or listener misconfiguration."

Scenario 5: ALB Path-Based Routing Is Not Working

Interview Question

Your ALB is configured for path-based routing (e.g., /api, /app), but requests are going to the wrong Target Group. How would you troubleshoot it?

Answer

If path-based routing is not working, I verify the listener rules, path patterns, priorities, and Target Groups.

Step 1: Check Listener Rules

Verify the ALB listener has the correct rules.

Example:

```
/api/* → API Target Group  
/app/* → App Target Group
```

Step 2: Check Rule Priority

ALB evaluates rules in **priority order**.

Ensure a generic rule like:

```
/*
```

doesn't have a higher priority than:

```
/api/*
```

Otherwise, all requests will match the generic rule first.

Step 3: Verify Target Groups

Ensure each listener rule forwards traffic to the correct Target Group and that the targets are **Healthy**.

Step 4: Test Routing

Verify routing using:

```
curl http://<ALB-DNS>/api/users  
curl http://<ALB-DNS>/app/login
```

Confirm each request reaches the correct backend service.

Step 5: Review Logs

Check:

- ALB Access Logs
- Application logs

Verify which Target Group received the request.

Real-Time Scenario

A microservices application had two services:

- `/api/*`
- `/admin/*`

After a deployment, all requests were routed to the API service. Investigation showed the default rule (`/*`) had a **higher priority** than `/admin/*`. After correcting the listener rule priority, routing worked as expected.

Best Practices

- Assign unique priorities to listener rules.
 - Keep the default rule at the lowest priority.
 - Test routing after every deployment.
 - Monitor ALB Access Logs.
-

Interview Summary

"If ALB path-based routing is not working, I first verify the listener rules and their priorities, ensure requests are forwarded to the correct Target Group, confirm the targets are healthy, test routing using curl, and review ALB Access Logs to identify any misconfigured rules."

💡 Interview Tip

A common follow-up question is:

What is the difference between Path-Based Routing and Host-Based Routing?

Path-Based Routing	Host-Based Routing
Routes based on URL path (<code>/api</code> , <code>/admin</code>)	Routes based on hostname (<code>api.example.com</code> , <code>app.example.com</code>)
Best for microservices	Best for multiple websites/domains
Uses one domain with different paths	Uses different domains or subdomains

Scenario 6: Sticky Sessions Are Not Working

Interview Question

Your application requires sticky sessions, but users are being routed to different EC2 instances on each request. How would you troubleshoot it?

Answer

If sticky sessions are not working, I verify the Target Group configuration, application cookies, and load balancer settings.

Step 1: Check Sticky Session Configuration

Verify that **Stickiness** is enabled on the Target Group.

Also check:

- Cookie type
- Cookie duration

Step 2: Verify Cookies

Open the browser's Developer Tools or use:

```
curl -I http://<ALB-DNS>
```

Ensure the response contains:

AWSALB

or

AWSALBTG

If no cookie is returned, stickiness is not working.

Step 3: Check Target Health

Ensure all backend targets are **Healthy**.

If an instance becomes unhealthy, the ALB routes traffic to another healthy instance, even with stickiness enabled.

Step 4: Verify Application Behavior

Ensure the application isn't clearing or overwriting the ALB cookie.

Also verify that application sessions are configured correctly.

Step 5: Test the Session

Send multiple requests from the same client and confirm they are served by the same backend instance.

Real-Time Scenario

A legacy web application required session persistence. Users were randomly logged out because **Target Group stickiness was disabled** after a deployment. After enabling stickiness and setting an appropriate cookie duration, users consistently reached the same backend server.

Best Practices

- Use sticky sessions only for **stateful applications**.
 - For modern applications, prefer **stateless architecture** with shared session storage (Redis, DynamoDB, etc.).
 - Monitor Target Group health to avoid unexpected failovers.
-

Interview Summary

"If sticky sessions are not working, I first verify that stickiness is enabled on the Target Group, check that the ALB cookie is being returned, ensure the backend targets are healthy, verify the application isn't overwriting the cookie, and test multiple requests to confirm session persistence."

💡 Interview Tip

A common follow-up question is:

When should you avoid sticky sessions?

Answer:

- Microservices architecture
- Auto Scaling environments
- Kubernetes/EKS deployments
- Applications using Redis, DynamoDB, or database-backed sessions

In these cases, **stateless applications** are preferred because any instance can handle any request.

Scenario 7: High Latency Through ALB

Interview Question

Users report that the application is very slow, even though all EC2 instances behind the ALB are healthy. How would you troubleshoot it?

Answer

If users experience high latency, I determine whether the delay is caused by the ALB, the application, or the backend infrastructure.

Step 1: Check CloudWatch Metrics

Review:

- **TargetResponseTime**
- **RequestCount**
- **HTTPCode_ELB_5XX**
- **HTTPCode_Target_5XX**

High **TargetResponseTime** usually indicates a slow backend application.

Step 2: Check EC2 Resources

Verify CPU, memory, and disk usage.

```
top  
free -m
```

df -h

High resource usage can slow application responses.

Step 3: Check Application & Database

Verify:

- Application logs
- Database performance
- External API calls

Slow SQL queries or external API timeouts are common causes of latency.

Step 4: Review ALB Access Logs

Check ALB Access Logs for:

- Request processing time
- Target processing time
- Response processing time

This helps identify where the delay occurs.

Step 5: Scale the Application

If the application is under heavy load:

- Increase the number of EC2 instances using Auto Scaling.
 - Upgrade the instance type if required.
-

Real-Time Scenario

An e-commerce application became slow during a sale. CloudWatch showed **TargetResponseTime** increasing significantly, while EC2 CPU utilization remained above 95%. After Auto Scaling launched additional instances, response times returned to normal.

Best Practices

- Monitor CloudWatch metrics and set alarms.
 - Use Auto Scaling for traffic spikes.
 - Optimize database queries.
 - Enable ALB Access Logs for performance analysis.
-

Interview Summary

"If users experience high latency through an ALB, I first review CloudWatch metrics, especially TargetResponseTime. Then I check EC2 resource utilization, analyze application and database performance, review ALB Access Logs, and scale the application if required. In most cases, latency is caused by backend resource constraints or slow database queries rather than the ALB itself."

💡 Interview Tip

A common follow-up question is:

How do you identify whether the latency is caused by the ALB or the backend application?

Answer:

- **High TargetResponseTime** → Backend application is slow.
 - **High ELB 5XX errors** → Load Balancer issue.
 - **High Target 5XX errors** → Application issue.
 - **ALB Access Logs** help pinpoint where the request spends the most time.
-

Scenario 8: NLB Cross-Zone Load Balancing Is Not Working

Interview Question

Your application is deployed across multiple Availability Zones, but traffic is not being distributed evenly by the Network Load Balancer (NLB). How would you troubleshoot it?

Answer

If traffic is uneven across Availability Zones, I verify the NLB configuration, target registration, and Cross-Zone Load Balancing settings.

Step 1: Check Cross-Zone Load Balancing

Verify whether **Cross-Zone Load Balancing** is enabled on the NLB.

If disabled, each NLB node sends traffic only to targets in its own Availability Zone.

Step 2: Verify Target Registration

Ensure healthy targets are registered in **all Availability Zones**.

If one AZ has fewer or no healthy targets, traffic distribution will be uneven.

Step 3: Check Target Health

Verify all targets are **Healthy**.

Unhealthy targets reduce the number of available backend servers in an AZ.

Step 4: Review CloudWatch Metrics

Monitor:

- HealthyHostCount
- UnHealthyHostCount
- ActiveFlowCount
- NewFlowCount

These metrics help identify traffic imbalance.

Step 5: Verify Auto Scaling

Ensure the Auto Scaling Group is configured to launch instances across multiple Availability Zones.

Real-Time Scenario

A production application was deployed across **two Availability Zones**, but one AZ had no healthy EC2 instances due to failed health checks. As a result, the NLB sent most traffic to the other AZ. After fixing the health check issue and enabling Cross-Zone Load Balancing, traffic was distributed evenly.

Best Practices

- Enable **Cross-Zone Load Balancing** when appropriate.
 - Deploy backend instances across multiple AZs.
 - Monitor Target Health and CloudWatch metrics.
 - Use Auto Scaling across multiple AZs.
-

Interview Summary

"If an NLB is not distributing traffic evenly, I first verify whether **Cross-Zone Load Balancing** is enabled, ensure healthy targets exist in all **Availability Zones**, review **CloudWatch** metrics, and confirm the **Auto Scaling Group** spans multiple AZs. Uneven traffic is often caused by unhealthy targets or missing instances in one **Availability Zone**."

💡 Interview Tip

A common follow-up question is:

What is the difference between Cross-Zone Load Balancing in ALB and NLB?

ALB	NLB
Cross-Zone Load Balancing is enabled by default .	Cross-Zone Load Balancing is optional and can be enabled or disabled.
Distributes requests evenly across all healthy targets.	By default, traffic stays within the same AZ unless Cross-Zone is enabled.

Mentioning this difference is a great way to demonstrate deeper AWS networking knowledge.

Scenario 9: Zero-Downtime Deployment Using ALB

Interview Question

How would you deploy a new application version without any downtime using an ALB?

Answer

To achieve zero-downtime deployment, I use **ALB + Target Groups + Auto Scaling** so that users continue to access healthy instances during the deployment.

Step 1: Create a New Version

Deploy the new application version to a new set of EC2 instances or a new Auto Scaling Group.

Step 2: Verify Health Checks

Ensure the new instances pass the ALB health checks before receiving production traffic.

Only **Healthy** instances should receive requests.

Step 3: Shift Traffic

Update the ALB Target Group or Listener Rule to gradually route traffic to the new version.

Once verified, direct all traffic to the new instances.

Step 4: Monitor the Deployment

Monitor:

- CloudWatch Metrics
- ALB Target Health
- Application Logs
- HTTP 5XX Errors

If issues are detected, stop the deployment.

Step 5: Roll Back (if Required)

If the new version fails:

- Redirect traffic back to the previous Target Group.
 - Investigate and fix the issue before redeploying.
-

Real-Time Scenario

During a production deployment, a new version of the application was deployed to a new Auto Scaling Group. After all instances became **Healthy**, the ALB was updated to route traffic to the new Target Group. When an issue was detected, traffic was immediately switched back to the previous Target Group, resulting in **zero downtime** for users.

Best Practices

- Use **Blue-Green** or **Rolling Deployments**.
 - Never route traffic to unhealthy instances.
 - Monitor CloudWatch and ALB metrics during deployment.
 - Always have a rollback plan.
-

Interview Summary

"For zero-downtime deployment, I deploy the new version to separate instances, verify they pass ALB health checks, gradually shift traffic using Target Groups, monitor the deployment, and roll back immediately if any issues occur. This ensures users experience no downtime during the release."

💡 Interview Tip

A common follow-up question is:

Which deployment strategies can be used with an ALB?

- **Blue-Green Deployment** – Switch traffic from the old environment to the new one.
- **Rolling Deployment** – Replace instances gradually.
- **Canary Deployment** – Route a small percentage of traffic to the new version before full rollout.

Mentioning these deployment strategies shows practical DevOps experience.

Scenario 10: Design a Highly Available ALB/NLB Architecture

Interview Question

How would you design a highly available, scalable, and secure architecture using ALB/NLB for a production application?

Answer

For a production environment, I design the architecture to ensure **high availability, scalability, fault tolerance, and security**.

Step 1: Deploy Across Multiple Availability Zones

Deploy the Load Balancer and backend instances across **at least two Availability Zones**.

This ensures the application remains available even if one AZ fails.

Step 2: Choose the Right Load Balancer

- **ALB** → HTTP/HTTPS applications, microservices, path-based and host-based routing.
 - **NLB** → TCP/UDP/TLS applications, low latency, high throughput, databases.
-

Step 3: Configure Auto Scaling

Deploy EC2 instances in an **Auto Scaling Group** across multiple AZs.

This automatically replaces unhealthy instances and scales during traffic spikes.

Step 4: Configure Health Checks

Use dedicated health check endpoints such as:

```
/health
```

Only healthy targets should receive traffic.

Step 5: Secure the Environment

- Use **HTTPS** with ACM certificates.
 - Restrict backend access so only the Load Balancer can communicate with EC2.
 - Enable **AWS WAF** (for ALB) to protect against common web attacks.
-

Step 6: Monitoring & Logging

Enable:

- CloudWatch Metrics
- ALB/NLB Access Logs
- CloudTrail

Set alarms for:

- High latency
 - 5XX errors
 - Unhealthy targets
-

Real-Time Scenario

An e-commerce application was designed with an **internet-facing ALB** across **two Availability Zones**, an **Auto Scaling Group** for web servers in private subnets, and an **RDS Multi-AZ** database. During an AZ outage, the ALB automatically routed traffic to healthy instances in the second AZ, and Auto Scaling launched replacement instances, ensuring uninterrupted service.

Best Practices

- Deploy resources across multiple AZs.
 - Use Auto Scaling with ALB/NLB.
 - Enable HTTPS and AWS WAF.
 - Configure CloudWatch alarms and Access Logs.
 - Use separate Target Groups for different services.
-

Interview Summary

"For a production architecture, I deploy the Load Balancer and Auto Scaling Group across multiple Availability Zones, choose ALB or NLB based on the application type,

configure health checks, secure traffic using HTTPS and WAF, and enable monitoring with CloudWatch and Access Logs. This provides high availability, scalability, and fault tolerance."

💡 Interview Tips

When should you use ALB?

- Web applications (HTTP/HTTPS)
- Microservices
- Path-based routing (/api, /admin)
- Host-based routing (api.example.com)
- WebSockets and HTTP/2 support

When should you use NLB?

- TCP/UDP applications
- Databases (MySQL, PostgreSQL)
- Gaming applications
- IoT/MQTT workloads
- Very low latency and high throughput

ALB vs NLB (Quick Revision)

Feature	ALB	NLB
Layer	Layer 7	Layer 4
Protocols	HTTP/HTTPS	TCP/UDP/TLS
Routing	Path & Host Based	Port Based
WAF Support	✔ Yes	✘ No
Sticky Sessions	✔ Yes	✘ No
Best For	Web Apps, APIs	Databases, TCP Apps
